HookFlow Product Requirements Document (PRD)

Version: 1.2 · **Status:** Public beta ready (Milestones 1–3 complete; Milestone 4 next) · **Last updated:** 2026-11-01
Product type: Developer Infrastructure / Backend SaaS
Primary objective: Build a reliable, secure, and usable webhook delivery platform.

What changed in v1.2

Milestone 3 is complete. Changes are marked **Status (v1.2)** or **Decision (v1.2)** in the text.

- SSRF: connections are pinned to the checked address, which blocks DNS rebinding (13.1).
- Rate limiting and quotas: per-account ingestion limits with response headers, and per-endpoint concurrency limits (FR-11, 13.4).
- Monitoring: Prometheus metrics and logged operator alerts (FR-13).
- Retention decided: events 30 days, audit log 1 year (section 18).
- Deployment decided: Render for the backend, Vercel for the dashboard, GitHub Actions for CI (section 18).
- Database-outage, DNS-failure and load tests (section 14).
- Both known issues fixed.

Operating details (settings, metrics, alerts, capacity) are in `docs/operations.md`.

What changed in v1.1

Version 1.0 was written before implementation. Version 1.1 keeps its requirements and records the decisions made while building Milestones 1 and 2. Changes are marked **Decision (v1.1)** or **Status (v1.1)** in the text.

- Answered most of the open questions in section 18: wildcard subscriptions, endpoint deletion, and customisable retries.
- Made the default retry policy concrete (FR-06), and defined the signature header and secret-rotation behaviour (FR-07).
- Defined how replay works (FR-09), and kept both replay endpoints with different scopes (section 8).
- Defined how idempotency keys work (FR-04).
- Expanded the API specification to the full v1 surface.
- Added three entities to the data model: sessions, idempotency keys, and delivery replays. Added FR-14 (audit log).
- Added replay, cancellation, and crash-recovery transitions to the delivery state machine.
- Added implementation status to the release plan, plus a list of known issues.

The detailed API contract lives in `docs/api.md`; this document stays at the requirements level.

---

1. Executive Summary

HookFlow is a developer-facing platform that enables applications to send events to external HTTP endpoints reliably.

Developers integrating webhooks into their applications often need to implement:

- delivery queues
- retry policies
- request signing
- failure handling
- event logging
- monitoring

Building these capabilities independently consumes engineering time and creates opportunities for reliability and security problems.

HookFlow provides these capabilities through a unified API and dashboard. Developers can:

- register webhook endpoints
- publish events
- monitor delivery attempts
- configure retry behavior
- replay failed deliveries

HookFlow processes deliveries asynchronously and maintains a persistent record of each event's delivery history.

The platform will initially target independent developers, small SaaS companies, and engineering teams that need webhook infrastructure without maintaining their own delivery systems.

1.1 Product Vision

Make reliable webhook delivery accessible to developers through a simple API and a transparent, observable delivery pipeline.

1.2 Core Value Proposition

Send events once to HookFlow; let HookFlow manage delivery, retries, and observability.

The platform will not promise that every destination processes every event exactly once. Instead, it will provide:

- durable event records
- at-least-once delivery attempts under documented operating assumptions
- duplicate-aware processing
- transparent delivery status

2. Problem Statement

Applications frequently need to notify other applications when important events occur. Examples include:

- An order is created.
- A payment succeeds.
- A document finishes processing.
- A user account is created.
- A subscription expires.

A basic implementation sends an HTTP request directly to the receiving application. However, the destination might:

- be temporarily unavailable
- respond slowly
- return an error
- accept a request that the sender subsequently fails to acknowledge internally

Developers must then solve several problems:

Problem	Consequence
Destination unavailable	Notifications fail
No retry mechanism	Temporary failures become permanent
Duplicate requests	Business operations may execute multiple times
Missing delivery history	Developers cannot diagnose failures
No request authentication	Receivers cannot reliably verify authenticity
Blocking HTTP requests	Application latency increases
No replay functionality	Recovery requires manual intervention
No monitoring	Persistent delivery failures go unnoticed

HookFlow addresses these problems through a centralized webhook delivery service.

3. Product Goals and Non-Goals

3.1 Goals

The initial product must:

1. Allow users to create accounts and manage API credentials.
2. Allow users to register and configure webhook endpoints.
3. Accept event submissions through an authenticated REST API.

4. Persist events before acknowledging successful ingestion.
5. Deliver events asynchronously to registered destinations.
6. Retry eligible failed deliveries using configurable policies.
7. Sign outgoing requests so receivers can verify authenticity.
8. Maintain detailed delivery-attempt history.
9. Allow users to inspect and replay failed deliveries.
10. Provide a dashboard for managing endpoints, events, and deliveries.
11. Enforce tenant isolation, authentication, and outbound-request security.
12. Provide automated tests, API documentation, monitoring, and deployment configuration.

3.2 Non-Goals for the Initial Release

The initial release will not attempt to provide:

- A general-purpose event streaming platform.
- Guaranteed exactly-once delivery or processing.
- Kafka-based distributed streaming.
- A Kubernetes-based deployment architecture.
- Arbitrary workflow orchestration.
- A visual workflow builder.
- A marketplace of third-party integrations.
- A mobile application.
- Complex enterprise billing and subscription management.
- Guaranteed global delivery or multi-region disaster recovery.

These features may be considered after validating the initial product.

4. Target Users and Personas

4.1 Independent Developer

Profile: A developer building a personal project, API, or small SaaS application.

Needs:

- A straightforward API.
- Simple endpoint registration.
- Clear documentation.
- Visibility into delivery failures.
- Minimal infrastructure maintenance.

Success scenario: The developer integrates HookFlow into an application and can inspect the delivery history of every submitted event.

4.2 Startup Backend Engineer

Profile: An engineer maintaining multiple services that communicate through HTTP events.

Needs:

- Reliable asynchronous delivery.
- Configurable retries.
- Secure webhook signatures.
- Event replay.
- API-based endpoint management.
- Monitoring and debugging tools.

Success scenario: The engineer can recover from a temporary downstream outage without implementing a custom retry system.

4.3 Small SaaS Team

Profile: A small engineering team offering integrations to customers.

Needs:

- Multiple endpoints.
- Event subscriptions.
- Team access in a future release.
- Usage visibility.
- Delivery logs and incident diagnosis.

Success scenario: The team uses HookFlow to provide outbound webhooks to its own customers.

5. Product Scope

The product will consist of four primary components.

5.1 Developer API

Responsible for:

- Authenticating API requests.
- Managing endpoints.
- Accepting event submissions.
- Returning event identifiers.
- Exposing event and delivery status.
- Supporting delivery replay.

5.2 Delivery Engine

Responsible for:

- Scheduling deliveries.
- Executing outbound HTTP requests.
- Signing request payloads.
- Applying timeouts.
- Recording attempt outcomes.
- Scheduling retries.
- Recovering abandoned work.

5.3 Web Dashboard

Responsible for:

- Account and API key management.
- Endpoint configuration.
- Event browsing.
- Delivery inspection.
- Failure investigation.
- Manual replay.
- Basic usage analytics.

5.4 Background Processing and Observability

Responsible for:

- Queue management.
- Worker coordination.
- Delivery scheduling.
- Metrics collection.
- Structured logging.
- Operational health monitoring.

6. Functional Requirements

Priority definitions:

- **P0:** Required for the initial usable release.
- **P1:** Required for a robust public beta.
- **P2:** Suitable for a later release.

FR-01: User Authentication

Priority: P0. **Status (v1.1):** Done.

Users must be able to create accounts and authenticate securely.

Requirements:

- Support account registration and login.
- Store passwords using a suitable password-hashing algorithm.
- Support logout and session invalidation.
- Provide account-level data isolation.
- Protect authenticated dashboard routes.
- Apply request limits to authentication endpoints.

Acceptance criteria:

- Unauthenticated users cannot access protected resources.
- Users cannot retrieve another user's endpoints or events.
- Passwords are never stored in plaintext.
- Invalid credentials produce an appropriate authentication error.

> **Decision (v1.1):**
>
> - **Passwords:** hashed with Argon2, minimum 10 characters.
> - **Sessions:** server-side, sent as an HttpOnly, SameSite=Lax cookie. They last 7 days and are invalidated on logout.
> - **Rate limits:** login and register are limited to 10 per minute per IP and per email.
> - **Errors:** a wrong password and an unknown email return the same error, and take about the same time.

FR-02: API Key Management

Priority: P0. **Status (v1.1):** Done.

Users must be able to authenticate programmatic requests using API keys.

Requirements:

- Generate API keys securely.
- Display the complete key only when initially created.
- Store a secure key verifier rather than the plaintext key.
- Support revocation.
- Associate each key with its owner.
- Allow users to identify keys using a non-secret prefix.

Acceptance criteria:

- A valid key authorizes requests according to its permissions.
- An invalid or revoked key cannot authorize new requests.
- API keys are never written to application logs.
- API key ownership is enforced on every authenticated operation.

> **Decision (v1.1):**
>
> - **Format:** keys look like `hf_<8-hex prefix>_<secret>`. Only a SHA-256 hash and the prefix are stored, and the hash is compared in constant time.
> - **Management:** API keys can't create or revoke other API keys; that's dashboard (session) only.
> - **Permissions:** for now every key has the same full permissions over its account. Scoped keys are a later improvement.

FR-03: Endpoint Management

Priority: P0. **Status (v1.1):** Done.

Users must be able to register webhook destinations.

Requirements:

- Create an endpoint with a destination URL.
- Configure event subscriptions.
- Enable or disable an endpoint.
- View endpoint details.
- Update endpoint configuration.
- Delete or archive an endpoint.
- Generate and rotate signing secrets.

Acceptance criteria:

- An endpoint belongs to exactly one customer account.
- Only its authorized owner can modify its configuration.
- Disabled endpoints receive no new deliveries.
- Existing delivery history remains available according to the retention policy.

> **Decision (v1.1):**
>
> - **Subscriptions:** exact event types, plus `*` for every event type. Pattern wildcards such as `order.*` are not supported yet.
> - **Deleting:** this archives the endpoint. It disappears from lists and gets no new deliveries, its unsent deliveries are cancelled, and its delivery history is kept.
> - **Disabling:** this also cancels unsent deliveries. They can be replayed after the endpoint is re-enabled.
> - **Limits:** each account can have at most 50 endpoints.
> - **Retries:** each endpoint has its own `max_attempts` (see FR-06).

FR-04: Event Ingestion

Priority: P0. **Status (v1.1):** Done.

Applications must be able to publish events through a REST API.

Example request:

```
POST /v1/events
{
  "event_type": "order.created",
  "data": { "order_id": "ORD-101", "amount": 2499 }
}
```

Requirements:

- Authenticate the request.
- Validate the event type and JSON payload.
- Generate a unique event identifier.
- Persist the event and associated delivery work.
- Identify eligible subscribed endpoints.
- Return the event ID and processing status.
- Support idempotency keys.
- Enforce payload-size limits.

Example successful response (`202 Accepted`):

```
{ "event_id": "evt_12345", "status": "accepted", "created_at": "2026-10-30T12:00:00Z", "deliver
```

Acceptance criteria:

- A successful response is returned only after the event and its required delivery records have been durably persisted.
- The API does not wait for destination endpoints to respond.

- Repeating a request with the same idempotency key and equivalent parameters does not create another event.
- Reusing an idempotency key with a different payload returns a conflict.
- Events are delivered only to eligible endpoints belonging to the submitting account.

> **Decision (v1.1): idempotency keys**
>
> - **Header:** the key goes in an `Idempotency-Key` header of 1–255 printable ASCII characters.
> - **Scope:** keys are scoped to the account and remembered for 24 hours.
> - **Same key, equivalent request:** returns the original `202` response with `Idempotent-Replayed: true`, and nothing is created or queued. "Equivalent" means the same `event_type` and the same `data` compared as JSON values, so key order and whitespace don't matter.
> - **Same key, different payload:** `409 idempotency_key_reused`, with the original event ID in the error details.
> - **Concurrent requests:** two requests racing with the same new key create exactly one event (enforced by a unique constraint).
>
> **Decision (v1.1): validation**
>
> - **Event type:** 1–100 characters; letters, digits, `_` and `-` in dot-separated parts.
> - **Data:** must be a JSON object.
> - **Size:** the body is limited to 1 MiB (`413` if larger).
> - **Response:** includes `delivery_count`, the number of endpoints the event will be delivered to.

FR-05: Asynchronous Delivery

Priority: P0. **Status (v1.1):** Done.

HookFlow must deliver events independently of the ingestion request.

Requirements:

- Queue delivery work.
- Execute outbound HTTP POST requests.
- Include the event ID, event type, timestamp, and event data.
- Apply configurable connection and response timeouts.
- Record the result of every attempt.
- Support concurrent workers.
- Prevent simultaneous workers from independently claiming the same active delivery.

Acceptance criteria:

- A successful destination response marks the delivery as delivered.
- An unsuccessful attempt is recorded.
- One endpoint's failure does not prevent delivery to other endpoints.
- Worker termination does not permanently lose durably recorded delivery work.

- Recovery logic can identify and reclaim abandoned processing jobs safely.

> **Decision (v1.1): what counts as success**
>
> - Any 2xx response is a success.
> - Redirects (3xx) are never followed. They count as a permanent failure, with a message asking the user to update the URL.
>
> **Decision (v1.1): timeouts and limits**
>
> - 10 seconds per attempt and 5 seconds to connect; both are configurable.
> - Only the first 4 KiB of each response body is stored.
>
> **Decision (v1.1): request body**
>
> - `{"id", "type", "created_at", "data"}`.
> - Headers: `HookFlow-Event-Id`, `-Event-Type`, `-Delivery-Id`, `-Attempt`, `-Timestamp` and `-Signature`.
>
> **Decision (v1.1): claims and leases**
>
> - A worker claims a delivery with one atomic conditional `UPDATE`, which also takes a 60-second lease.
> - A sweeper runs every 5 seconds. It reclaims deliveries whose lease expired and records the lost attempt as "worker stopped; the destination may or may not have received it".
> - A late result from a worker that lost its lease is kept in the attempt history, but it doesn't overwrite the delivery's state.

FR-06: Automatic Retries

Priority: P0. **Status (v1.1):** Done.

HookFlow must retry eligible failed deliveries.

Requirements:

- Configure maximum attempts.
- Support exponential backoff.
- Add randomized jitter to retry delays.
- Distinguish retryable from non-retryable responses.
- Persist the next scheduled attempt.
- Stop automatic retries when the configured limit is reached.

Default policy for the initial release:

- Maximum attempts: 5, including the initial attempt.
- Retryable responses: HTTP 429, selected HTTP 5xx responses, connection failures, and timeouts.
- Other HTTP 4xx responses: generally non-retryable.

- Retry delays: configurable, with exponential backoff and jitter.

Acceptance criteria:

- Retry scheduling survives application restarts.
- A delivery is not retried before its scheduled time.
- The attempt count is accurate.
- Exhausted deliveries become eligible for manual replay.
- Operators can distinguish permanent failures from scheduled retries.

> **Decision (v1.1): what is retried**
>
> - **Retried:** HTTP 408, 429 and all 5xx responses, plus connection errors and timeouts.
> - **Not retried:** all other 4xx responses and 3xx redirects. They fail at once.
>
> **Decision (v1.1): delays**
>
> - The delay after attempt *n* is $30s \times 4^{n-1}$ with ±25% jitter, capped at 6 hours. That gives about 30s, 2m, 8m, 32m, 2h.
> - A `Retry-After` header from the destination is honoured, up to the same cap.
> - The base delay and the cap are set by environment variables.
>
> **Decision (v1.1): attempt limits**
>
> - Each endpoint has its own `max_attempts`, from 1 to 10 (default 5). This answers the open question in section 18.
> - A delivery takes its limit when it is created, so changing the limit only affects new deliveries.
> - Fully custom per-endpoint retry schedules are left to Milestone 4.
>
> **Decision (v1.1): visibility**
>
> - Scheduled retries are in the `retrying` state, with `next_attempt_at` set.
> - Permanent failures are in the `failed` state, with a readable reason such as "Destination returned HTTP 503. Gave up after 5 attempts."

FR-07: Request Signing

Priority: P0. **Status (v1.1):** Done.

Outgoing webhook requests must be verifiable by receiving applications.

Requirements:

- Use HMAC-SHA256.
- Include a request timestamp.
- Sign the timestamp and exact serialized request body.
- Provide the signature in a dedicated HTTP header.

- Support secret rotation.
- Document receiver-side verification.

Acceptance criteria:

- The receiver can independently verify a valid signature.
- Changing the signed body invalidates the signature.
- Invalid signatures can be detected.
- Secret values are not exposed in delivery logs.
- Documentation explains timestamp validation and replay protection.

> **Decision (v1.1): signature format**
>
> - The header is `HookFlow-Signature: v1=<hex HMAC-SHA256 of "{timestamp}.{raw body}">`.
> - The timestamp is sent in `HookFlow-Timestamp`.
> - Receivers should reject timestamps more than 5 minutes off their clock.
>
> **Decision (v1.1): secrets**
>
> - Secrets look like `whsec_…` and are encrypted at rest.
> - Every time a secret is revealed, it is recorded in the audit log.
>
> **Decision (v1.1): rotation**
>
> - Rotation has a grace period: 24 hours by default, configurable from 0 to 7 days.
> - During the grace period both secrets sign each request, `v1=<new>,v1=<old>`, so receivers can switch at any time without rejecting requests.
> - Receivers accept a request if any `v1=` value matches.

FR-08: Delivery History

Priority: P0. **Status (v1.1):** Done.

Users must be able to inspect delivery attempts.

Requirements:

- Show event ID and endpoint.
- Show delivery status.
- Show attempt count.
- Record HTTP response status.
- Record attempt start and completion timestamps.
- Record response latency.
- Store sanitized error details.
- Show the next scheduled retry, when applicable.

Acceptance criteria:

- Every completed attempt has a corresponding history record.
- Failed attempts remain visible after a later successful retry.
- Users cannot view another customer's delivery records.
- Response-body storage is bounded and sensitive information is handled safely.

FR-09: Manual Replay

Priority: P0. **Status (v1.1):** Done.

Users must be able to request a new delivery attempt for an eligible failed delivery.

Requirements:

- Provide a replay action in the dashboard.
- Provide an authenticated replay API.
- Preserve the original event ID.
- Maintain a history of replay operations.
- Apply current endpoint security and delivery limits.
- Prevent unauthorized replay operations.

Acceptance criteria:

- Replay creates an auditable delivery operation.
- The original attempt history remains intact.
- Replay does not silently create a new business event.
- Repeated replay requests cannot bypass rate limits or cause uncontrolled delivery storms.

> **Decision (v1.1): what a replay does**
>
> - It resets the same delivery to `pending`, keeping the same event ID, delivery ID and body. No new event or delivery is created.
> - Earlier attempts are kept, and new attempts continue the numbering.
> - The replay gets a fresh retry budget equal to the endpoint's `max_attempts`, with normal backoff.
>
> **Decision (v1.1): what can be replayed**
>
> - Only `failed` and `cancelled` deliveries.
> - The endpoint must be enabled and not deleted, and its current URL must pass the current URL rules.
> - The current URL and secret are used, so a user can fix a wrong URL and then replay.
>
> **Decision (v1.1): storm protection**
>
> - A replayed delivery is `pending` again, so repeating the request returns `409` and can't stack up.
> - Replays are rate limited to 60 deliveries per minute per account.
>
> **Decision (v1.1): history**
>
> - Each replay is stored with who requested it (dashboard session or which API key), the previous status, and the first attempt number it covers.
> - Each replay is also written to the audit log.
>
> **Decision (v1.1): endpoints**
>
> - Both replay endpoints are kept: one replays a single delivery, the other replays an event's failed deliveries (see section 8).

FR-10: Dashboard and Analytics

Priority: P0. **Status (v1.1):** Done.

The dashboard must provide the core workflows needed to use the product.

Requirements:

- Display total events and deliveries.
- Show successful, pending, retrying, and failed deliveries.
- List and filter events.
- List and configure endpoints.
- Inspect delivery history.
- Trigger eligible replays.
- Display basic time-range-based metrics.

Acceptance criteria:

- Users can navigate from an event to its deliveries.
- Users can identify why a delivery failed.
- Filters work across relevant statuses and dates.
- Dashboard data is scoped to the authenticated account.

> **Status (v1.1):**
>
> - **Time ranges:** 24h, 7d and 30d, with an events and deliveries chart.
> - **Onboarding:** an onboarding checklist.
> - **Pages:** events, deliveries, endpoints, API keys and audit log.
> - **Replay:** confirmation before replaying, and replay markers in the attempt history.

FR-11: Rate Limiting and Quotas

Priority: P1. **Status (v1.2):** Done.

Requirements:

- Apply limits to event ingestion.
- Apply limits to authentication endpoints.
- Configure per-account quotas.
- Limit outbound delivery concurrency.
- Return HTTP 429 when applicable.
- Expose appropriate rate-limit information.

> **Status (v1.2):**
>
> - **Ingestion:** 600 events per minute and 10,000 new events per UTC day per account by default. Either can be changed per account; `0` means unlimited.
> - **Responses:** `X-RateLimit-*` and `X-Quota-*` headers on every ingestion response. `429 rate_limited` or `429 quota_exceeded`, with `Retry-After`. Idempotent retries don't use quota.
> - **Auth and replay:** login and registration are limited per IP and per email; replays per account.
> - **Outbound:** at most 10 requests in flight per endpoint across all workers. Extra deliveries wait without using an attempt.
> - **Redis down:** limits fail open, so a Redis outage doesn't block ingestion.

FR-12: Notifications

Priority: P1. **Status (v1.1):** Not started.

Requirements:

- Allow users to enable delivery-failure notifications.
- Notify users when configured failure thresholds are reached.

- Avoid repeated notifications for the same incident.
- Track notification failures separately from webhook delivery failures.

FR-13: Operational Monitoring

Priority: P1. **Status (v1.2):** Done.

Requirements:

- Expose API latency and error metrics.
- Track queue depth.
- Track worker health.
- Measure delivery latency and success rates.
- Emit structured logs.
- Provide health-check endpoints.
- Alert operators to persistent processing failures.

> **Status (v1.2):**
>
> - **Metrics:** `GET /metrics` serves Prometheus metrics: API request counts and latency, queue depth, the oldest waiting delivery, outbox backlog, recent attempt success and latency, and worker heartbeat. It can be protected with a token.
> - **Alerts:** a monitor task logs an `ALERT` line when due work or queue messages wait over 5 minutes. Suggested Prometheus rules cover worker-down and failure-rate alerts.
> - **Health:** `/health/ready` also reports whether the worker is running.

FR-14: Audit Log (added in v1.1)

Priority: P0. **Status (v1.1):** Done.

Section 9 already listed audit logs as an entity and FR-09 requires replays to be auditable. This requirement makes the feature explicit.

Requirements:

- Record important account actions:
- registration and login
- API key creation and revocation
- endpoint creation, changes and archiving
- signing secrets being revealed or rotated
- replays
- Record who acted (dashboard session or a specific API key), when, from which IP, and what changed. For an endpoint change, that means each field's old and new value.
- Never store secrets, keys or passwords in audit entries.
- Let users list and filter their own audit log through the API and the dashboard.

7. Non-Functional Requirements

7.1 Reliability

- Persist accepted events and required delivery records before acknowledging ingestion.
- Recover unfinished work after worker crashes.
- Make queue publication recoverable.
- Ensure that delivery state transitions are concurrency-safe.
- Document at-least-once delivery semantics.
- Avoid claiming exactly-once delivery.

7.2 Performance

Initial engineering targets, to be validated through load testing:

Metric	Initial target
Event ingestion latency	p95 below 200 ms under defined test load
Event ingestion response	Independent of destination response time
Dashboard API latency	p95 below 300 ms under defined test load
Delivery concurrency	Configurable
Payload limit	1 MiB by default
Delivery timeout	Configurable; initially 10 seconds

These are development targets, not guarantees. Results must be measured against a documented deployment configuration and workload.

7.3 Security

- Enforce authentication and authorization.
- Isolate tenant data.
- Encrypt traffic in transit.
- Protect stored credentials and signing secrets.
- Validate incoming payloads.
- Protect against SSRF and DNS rebinding.
- Prevent unauthorized access to internal network resources.
- Limit response-body storage and payload sizes.
- Redact sensitive data from logs.
- Apply request and concurrency limits.

7.4 Scalability

The architecture should allow API instances and delivery workers to scale independently. The initial deployment does not require distributed multi-region infrastructure.

7.5 Maintainability

- Use modular application components.
- Provide versioned REST APIs.
- Manage schema changes with database migrations.
- Maintain automated tests.
- Document environment configuration.
- Use structured logging and consistent error responses.

7.6 Observability

The system must expose sufficient information to investigate:

- Missing deliveries.
- Repeated timeouts.
- Growing queues.
- Worker failures.
- Elevated API latency.
- Persistent endpoint outages.

8. API Specification

All endpoints use the `/v1` prefix. The full contract, including request and response shapes, is in `docs/api.md`.

Method	Endpoint	Purpose
POST	`/v1/events`	Publish an event (optional `Idempotency-Key` header)
GET	`/v1/events`	List events (filter by type and date)
GET	`/v1/events/{event_id}`	Retrieve an event
GET	`/v1/events/{event_id}/deliveries`	List event deliveries
POST	`/v1/events/{event_id}/replay`	Replay the event's failed or cancelled deliveries
POST	`/v1/endpoints`	Register an endpoint
GET	`/v1/endpoints`	List endpoints
GET	`/v1/endpoints/{endpoint_id}`	Retrieve endpoint details

Method	Endpoint	Purpose
PATCH	`/v1/endpoints/{endpoint_id}`	Update endpoint configuration
DELETE	`/v1/endpoints/{endpoint_id}`	Archive an endpoint
GET	`/v1/endpoints/{endpoint_id}/secret`	Reveal the signing secret *(added)*
POST	`/v1/endpoints/{endpoint_id}/rotate-secret`	Rotate the signing secret, with a grace period
GET	`/v1/deliveries`	List deliveries (filter by status, endpoint, event) *(added)*
GET	`/v1/deliveries/{delivery_id}`	Retrieve delivery status
GET	`/v1/deliveries/{delivery_id}/attempts`	Retrieve attempt history
POST	`/v1/deliveries/{delivery_id}/replay`	Replay one failed or cancelled delivery
GET	`/v1/deliveries/{delivery_id}/replays`	Replay history *(added)*
GET	`/v1/stats`	Counts and time series for the dashboard *(added)*
GET	`/v1/audit-logs`	Audit log *(added)*
POST	`/v1/auth/register`, `/login`, `/logout`; GET `/v1/auth/me`	Dashboard sessions *(added)*
GET, POST, DELETE	`/v1/api-keys`	API key management, dashboard only *(added)*
GET	`/health/live`, `/health/ready`	Liveness and readiness

> **Decision (v1.1):**
>
> - **Replay endpoints:** both are kept rather than merged into one. The delivery endpoint replays one delivery; the event endpoint replays every eligible delivery of the event, optionally limited by `endpoint_ids`. Both follow the same rules (FR-09).
> - **Pagination:** list endpoints use newest-first cursor pagination (`limit`, `cursor`, `next_cursor`).
> - **Errors:** every error uses `{"error": {"code", "message", "details"}}`.
> - **Not found:** another account's resources return `404`, the same as resources that don't exist.

9. Data Model

The database schema includes the following entities:

1. **Users:** Account information.
2. **Sessions:** Dashboard login sessions *(added)*.
3. **API Keys:** Authentication credentials and revocation metadata.

4. **Endpoints:** Destination URLs and endpoint configuration, including retry limit and rotation state.
5. **Endpoint Subscriptions:** Event types accepted by each endpoint.
6. **Events:** Immutable event payloads and metadata.
7. **Idempotency Keys:** Per-account keys with a request fingerprint and expiry *(added)*.
8. **Deliveries:** Delivery state for each event-endpoint pair.
9. **Delivery Attempts:** Individual HTTP attempt records.
10. **Delivery Replays:** One record per manual replay *(added)*.
11. **Outbox Events:** Durable records awaiting publication to the queue.
12. **Audit Logs:** Important user and administrative actions.

Important relationships:

- One user can own multiple API keys.
- One user can register multiple endpoints.
- One event can produce multiple deliveries.
- One delivery can have multiple attempts.
- One delivery can have multiple replays *(added)*.
- One endpoint can subscribe to multiple event types.
- Each delivery belongs to an event and an endpoint.

Data consistency rules:

- Event IDs must be unique.
- Idempotency keys must be unique within their defined customer scope (the account).
- Delivery claims must be atomic.
- Attempt history must be append-oriented.
- Event ownership and endpoint ownership must be validated.
- Durable delivery work must remain recoverable if queue publication fails.

> **Decision (v1.1):**
>
> - **IDs:** prefixed and time-sortable, such as `evt_…`, `dlv_…` and `ep_…`.
> - **Deliveries:** there is one per event-endpoint pair, enforced by a unique constraint.
> - **Housekeeping:** published outbox rows are deleted after 1 day and expired idempotency keys are removed. Event and attempt retention is still open (section 18).

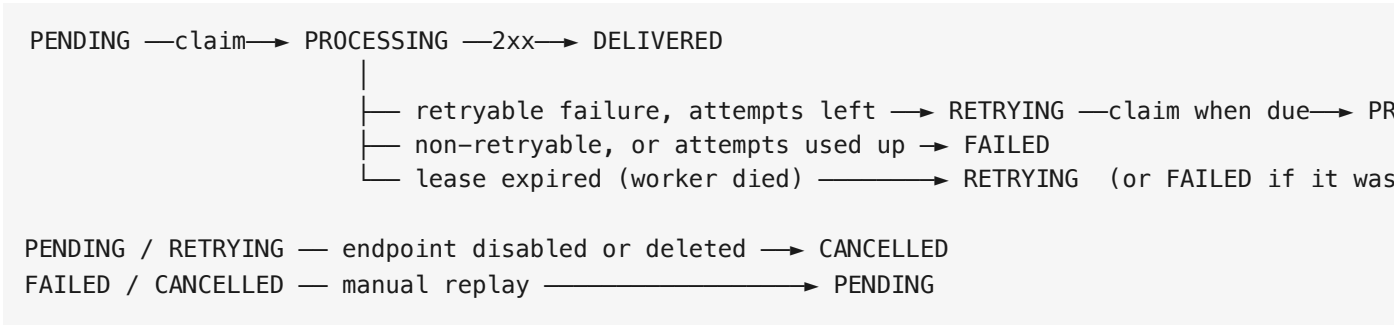
10. Delivery State Machine

A delivery must follow a well-defined lifecycle.

States:

- `pending`
- `processing`
- `retrying`

- `delivered`
- `failed`
- `cancelled`



From	To	When
pending, retrying	processing	A worker claims it, once `next_attempt_at` has passed (atomic, takes a lease)
processing	delivered	The destination returns 2xx
processing	retrying	A retryable failure with attempts left, or the lease expired
processing	failed	A non-retryable failure, or the attempt budget is used up
pending, retrying	cancelled	The endpoint is disabled or deleted
failed, cancelled	pending	Manual replay (FR-09)
delivered	—	Final

If a retryable failure exhausts the configured attempt budget, the delivery transitions to `failed`. State transitions must be validated and persisted transactionally.

> **Decision (v1.1): cancellation**
>
> - Disabling or deleting an endpoint cancels its `pending` and `retrying` deliveries.
> - A delivery that is already `processing` is not interrupted. Its attempt finishes and is recorded.
> - If a worker claims a delivery whose endpoint has since been disabled, it cancels the delivery without sending.
>
> **Decision (v1.1): crash recovery**
>
> - A `processing` delivery whose lease expires goes back to `retrying`, or to `failed` if that was its last attempt.
>
> **Decision (v1.1): replay**
>
> - `failed` and `cancelled` can go back to `pending` through a manual replay (FR-09).
> - `delivered` is final.
>
> **Decision (v1.1): how transitions are guarded**
>
> - Every transition is a conditional `UPDATE` on the expected current state, so a concurrent change makes it a no-op instead of corrupting the state.

11. User Experience Requirements

The product should be usable without requiring users to understand its internal queue architecture.

Onboarding:

1. Register an account.
2. Generate an API key.
3. Register a destination endpoint.
4. Configure event subscriptions.
5. Send a test event.
6. Inspect the delivery result.

Endpoint details page must display:

- Destination URL.
- Subscribed event types.
- Active or disabled state.
- Signing-secret management, including rotation grace status *(added)*.
- Retry limit (max attempts) *(added)*.
- Recent deliveries.
- Failure history.

Event details page must display:

- Event ID and event type.
- Original payload.
- Creation timestamp.
- Idempotency key, if one was used *(added)*.
- Matching endpoints.
- Delivery status for each endpoint.
- Attempt history and response details, with replay markers *(added)*.
- Replay action when permitted.

Error presentation: Errors should be actionable. For example, show: "Delivery failed: destination returned HTTP 503. The next retry is scheduled for 14:35 UTC." Do not expose internal stack traces, credentials, or sensitive infrastructure details.

12. Technical Architecture

Technology choices:

Layer	Technology
Frontend	Next.js and TypeScript
Backend API	FastAPI
Relational database	PostgreSQL
Background queue	Redis
Worker framework	Celery
ORM	SQLAlchemy
Migrations	Alembic
Testing	Pytest
Containers	Docker Compose
CI/CD	GitHub Actions
Monitoring	Prometheus and Grafana

Reliability architecture

- **Ingestion:** the API writes the event, matching delivery records, and outbox records in one PostgreSQL transaction.
- **Publishing:** an outbox publisher then enqueues pending delivery work.
- **Delivery:** workers claim deliveries using atomic database operations or leases, perform the HTTP request, and record the outcome.
- **Retries:** failed work is rescheduled according to its retry policy.

- **Crashes:** if a worker crashes after the destination receives a request but before success is recorded, the delivery may be attempted again. The platform must preserve the same event ID and document this at-least-once behavior.
- **Durability:** Redis coordinates processing but is not the sole durable record of accepted work.

> **Decision (v1.1): every queue message goes through the outbox**
>
> - This covers ingestion, scheduled retries (staged with a future `available_at`), replays and crash recovery. Each message is written in the same transaction as its work.
> - The API publishes right after commit to keep latency low.
> - A publisher runs every second and sends anything still unpublished. Events are accepted while Redis is down and are delivered once it's back.
> - Publishing a message twice is harmless, because the claim only succeeds once.
>
> **Decision (v1.1): sweeper**
>
> - A sweeper runs every 5 seconds. It reclaims expired leases, re-stages due work whose published message was lost in Redis, and cleans up old outbox rows.
>
> **Decision (v1.1): processes**
>
> - `api`, `worker` (as many as needed), `beat` (exactly one; it schedules the publisher and the sweeper), and the `migrate` job.

13. Security and Abuse Prevention

Webhook delivery introduces a critical security concern: the platform makes outbound HTTP requests to user-configured destinations. The following controls are mandatory before enabling unrestricted public endpoint registration.

13.1 SSRF Prevention

- Restrict destination URL schemes to HTTP and HTTPS. *(Done)*
- Reject loopback, private, link-local, reserved, and otherwise prohibited IP ranges. *(Done: at registration for literal IPs, and at send time after DNS resolution)*
- Validate DNS resolution and the destination IP. *(Done)*
- Defend against DNS rebinding and redirect-based bypasses. *(Done. Redirects are never followed. The connection uses the exact address that was checked, and one private address rejects the whole host)*
- Disable redirects by default. *(Done: redirects are never followed)*
- Enforce network-level egress restrictions. *(Documented in `docs/operations.md`. This is a deployment control; managed hosts like Render don't offer it)*
- Apply strict timeouts and response-size limits. *(Done)*

13.2 Credential Protection

- Hash API keys for verification.
- Encrypt signing secrets at rest.
- Restrict secret access. *(Every reveal is audited)*
- Redact credentials from logs.
- Support key and secret revocation or rotation.

13.3 Tenant Isolation

Every data-access operation must enforce ownership. Random or guessed resource IDs must not allow access to another tenant's records.

13.4 Resource Protection

- Limit event payload size. *(1 MiB)*
- Limit endpoint count per account. *(50)*
- Limit ingestion requests. *(Done: per-account rate limit and daily quota)*
- Limit concurrent outbound requests. *(Done: worker concurrency, plus 10 per endpoint)*
- Enforce queue and retry quotas. *(Attempts are capped at 10 per run, and replays are rate limited)*
- Apply retention and cleanup policies. *(Done: events and their history after 30 days, audit entries after 1 year, outbox messages after 1 day, idempotency keys after 24 hours)*

14. Testing Strategy

Unit testing. Test:

- Request validation.
- Signature generation and verification.
- Retry-delay calculations.
- Idempotency behavior.
- State-transition rules.
- Authorization checks.

Integration testing. Test:

- Database transactions.
- Event ingestion and outbox publication.
- Queue and worker interactions.
- Endpoint delivery.
- Retry scheduling.
- Secret rotation.
- Replay behavior.

Failure testing. Simulate:

- HTTP 500 responses.
- HTTP 429 responses.
- Connection timeouts.
- DNS failures.
- Worker termination.
- Database unavailability.
- Redis unavailability.
- Queue publication failures.
- Duplicate submissions.
- A destination processing a request before the worker crashes.

Security testing. Test:

- Unauthorized resource access.
- Cross-tenant access attempts.
- Forged signatures.
- SSRF attempts.
- Malicious redirect destinations.
- Oversized payloads.
- Rate-limit bypass attempts.
- Credential leakage through logs.

Load testing. Measure:

- Ingestion throughput.
- p95 and p99 ingestion latency.
- Delivery throughput.
- Queue growth.
- Worker recovery time.
- Database utilization.

Record the test environment and workload alongside every performance result.

> **Status (v1.1):** 86 automated tests pass.
>
> - **Covered:** unit, integration, failure, and most security cases. That includes concurrent idempotency, Redis down during ingestion, worker crash and lease recovery, late results after a lost lease, redirects, timeouts and tenant isolation on every route.
> - **Status (v1.2):** 118 automated tests pass. Added SSRF pinning and DNS-rebinding tests, DNS failures (retried), database outages (API returns 503, the worker recovers, metrics keep serving), limits and quotas, concurrency, retention, metrics and alerts.
> - **Load testing (v1.2):** see `docs/operations.md` for the setup and results. Ingestion p95 stays below 200 ms up to about 10 concurrent clients (about 160 events/s) on a single local API process. It reached 83 ms there.

15. Success Metrics

The product will be evaluated on usability, reliability, and engineering quality.

Product metrics:

- Number of registered users.
- Number of configured endpoints.
- Number of events accepted.
- Percentage of users who successfully complete a test delivery.
- Percentage of users who return and publish another event.
- Number of external developers actively using the platform.

Reliability metrics:

- Percentage of eligible deliveries eventually delivered within a defined observation window.
- Percentage of deliveries with complete attempt history.
- Number of lost accepted events.
- Queue processing delay.
- Worker recovery time.
- Rate of duplicate delivery attempts.

Engineering quality metrics:

- Automated test coverage of critical delivery paths.
- Successful recovery from worker crashes.
- Successful execution of security tests.
- API latency under defined workloads.
- Documented deployment and recovery procedures.

Targets for adoption and reliability should be established after the first usable deployment and initial testing.

16. Release Plan

Milestone 1: Core MVP (Week 1–2). Status (v1.1): Complete

Deliver:

- User authentication.
- API key management.
- Endpoint registration.
- Event ingestion.
- PostgreSQL persistence.
- Background delivery.
- Basic delivery history.
- Initial dashboard.

Exit criteria: A developer can register an endpoint, submit an event, receive it, and inspect its delivery result. *(Met)*

Milestone 2: Reliability (Week 3). Status (v1.1): Complete

Deliver:

- Configurable retries.
- Delivery-attempt history.
- HMAC signatures.
- Idempotency.
- Manual replay.
- Durable outbox processing.
- Worker recovery.

Also delivered (v1.1): an audit log (FR-14), a secret-rotation grace period, and per-endpoint retry limits.

Exit criteria: Transient failures can be retried, duplicate ingestion is handled correctly, and unfinished work can be recovered after a worker restart. *(Met. Recovery from a Redis outage is also verified.)*

Milestone 3: Public Beta Hardening (Week 4–5). Status (v1.2): Complete

Deliver:

- SSRF protections. *(Done: DNS-rebinding pinning; egress rules documented for self-managed networks)*
- Rate limiting. *(Done: auth, replay, ingestion, per-account quotas, per-endpoint concurrency)*
- Metrics and structured logging. *(Done: Prometheus metrics, JSON logs, logged alerts)*
- Integration tests. *(Done: 118 tests)*
- Load testing. *(Done: `loadtest/ingest.py`; results in `docs/operations.md`)*
- API documentation. *(Done: `docs/api.md`, `docs/operations.md` and interactive OpenAPI docs)*
- Production deployment. *(Done: Render and Vercel, with GitHub Actions CI)*
- Error handling and retention policies. *(Done: database outages return 503; 30-day event retention)*

Exit criteria: Critical security and recovery tests pass, operational metrics are available, and the system can be used safely within documented limits. *(Met.)*

Milestone 4: Product Improvements (Week 6 and beyond)

Potential additions:

- Email alerts.
- Team accounts.
- Usage analytics.
- SDKs.
- Endpoint health indicators.
- Advanced retry configuration (custom schedules per endpoint).
- Customer quotas.

- Pattern subscriptions such as `order.*` *(added)*.
- Scoped API keys *(added)*.
- Bulk replay, for example all failed deliveries for an endpoint in a time range *(added)*.

These are secondary to the reliability of the core delivery pipeline.

Known issues

Found during Milestones 1 and 2. **Status (v1.2):** both fixed.

- **Wrong audit IP for dashboard actions.** *Fixed:* `TRUSTED_PROXY_COUNT` tells HookFlow how many proxies to trust in `X-Forwarded-For`. This also fixes the per-IP login limit, which behind a proxy was shared by every user. The right value depends on the hosting setup, so check it after each deploy (see `docs/operations.md`).
- **Confusing attempt totals after replays.** *Fixed:* deliveries also report `run_attempt_count` and `run_max_attempts`, counted from the last replay. The dashboard shows those and keeps the lifetime total in a tooltip.

Remaining limitations:

- Request metrics are per API process; pipeline metrics are global.
- The monitor runs on the worker, so a stopped worker is detected by `/health/ready` or Prometheus, not by the monitor itself.
- Limits fail open when Redis is down.

17. Risks and Mitigations

Risk	Mitigation
Queue loses or delays work	Durable outbox and database-driven recovery
Worker crashes during delivery	Leases, recovery, and at-least-once processing
Duplicate delivery	Stable event IDs and receiver-side idempotency guidance
Duplicate ingestion *(added)*	Idempotency keys with conflict detection
SSRF attack	Destination validation, redirect controls, and egress restrictions
Unbounded retries	Maximum attempts, quotas, and backoff
Replay storms *(added)*	Replay only from final states, plus per-account replay rate limits
Database growth	Retention policies, indexing, and archival
Slow destinations	Strict timeouts and bounded concurrency
Credential leakage	Secret hashing or encryption, redaction, rotation, and audited secret access
Scope expansion	Release-based prioritization

Risk	Mitigation
No real users	Early deployment, developer feedback, and usable documentation

18. Assumptions and Open Questions

The PRD assumes:

1. HookFlow is a multi-tenant SaaS application.
2. The first release supports outbound HTTP webhooks.
3. Events use JSON payloads.
4. PostgreSQL is the durable system of record.
5. Redis is used for background processing coordination.
6. The initial deployment uses a single region.
7. Delivery semantics are at-least-once, not exactly-once.
8. The first version is operated by one developer.
9. A basic dashboard accompanies the API.
10. Advanced enterprise features are deferred.

Open questions and their status:

Question	Status (v1.1)
Should event subscriptions use exact event-type matching only, or support wildcard patterns?	**Decided:** exact matches plus `*` for all events. Pattern wildcards (`order.*`) are deferred to Milestone 4.
How long should events and delivery attempts be retained?	**Decided (v1.2):** 30 days for events with their deliveries and attempts (events with work still in progress are kept until it finishes), and 1 year for the audit log. Both are configurable.
Should endpoint deletion disable new deliveries immediately while preserving existing history?	**Decided: yes.** Deletion archives the endpoint, stops new deliveries, cancels unsent ones, and keeps history.
Should users be able to customize retry policies in the MVP?	**Decided: partly.** Per-endpoint maximum attempts (1–10). Backoff timing is set system-wide; custom schedules are deferred to Milestone 4.
Which deployment provider and object-storage options will be used?	**Decided (v1.2):** Render for the API, worker, Postgres and Redis, and Vercel for the dashboard, with GitHub Actions for CI. No object storage is needed yet: payloads and response bodies are capped and stored in Postgres.
Will the initial service be public, private, or limited to invited beta users?	**Recommendation (v1.2):** the SSRF and rate-limit work is done, so a public beta is possible. Keep the default limits, and set `METRICS_TOKEN` and `TRUSTED_PROXY_COUNT` first.

19. Definition of Done

HookFlow's first public beta is complete when:

Criterion	Status (v1.1)
Users can register and authenticate.	Done
API keys can be created and revoked.	Done
Users can register secure webhook endpoints.	Done
Events can be accepted and persisted.	Done
Matching delivery work can be recovered after queue or worker failures.	Done
Workers can deliver events asynchronously.	Done
Retry policies behave as documented.	Done
Requests include verifiable signatures.	Done
Idempotency is enforced for event ingestion.	Done
Delivery attempts are durably recorded.	Done
Failed deliveries can be inspected and replayed.	Done
Tenant isolation tests pass.	Done
SSRF protections are implemented and tested.	Partly (DNS rebinding and egress rules remain)
Critical workflows have automated integration tests.	Done
The application is deployable through documented configuration.	Partly (local Docker Compose; production deployment remains)
Health metrics and operational logs are available.	Partly (health checks and logs; metrics remain)
API documentation enables a new developer to complete a test delivery without assistance.	Done

20. Final Product Direction

HookFlow should begin as a focused webhook delivery service rather than a general-purpose distributed event platform. Its first job is to reliably accept an event, persist the work, deliver the event to subscribed endpoints, and explain exactly what happened.

The core product loop is:

Publish event → Persist event → Schedule delivery → Deliver → Record result → Retry if necessary → Inspect or replay.

Every advanced feature should strengthen that loop. The initial release should prioritize correctness, security, transparency, and ease of integration over the number of technologies used or the complexity of the architecture.

The product is successful when a developer can integrate HookFlow into a real application, trust its documented delivery behavior, and diagnose failures without needing to inspect the underlying infrastructure.